

UNIVERSITÀ DEGLI STUDI DI ROMA TRE

DIPARTIMENTO DI INGEGNERIA CIVILE, INFORMATICA E DELLE
TECNOLOGIE AERONAUTICHE



PROGETTO DI BIGDATA

EASY-QUERY: Un Sistema di AI-Agent per il Data Engineering Scalabile con Architettura Medallion

Autori

Luca Borrelli
Davide Mariani

Matricole

559443
566836

Sommario

EasyQuery è un sistema avanzato di gestione e interrogazione dei dati, progettato per semplificare l'interazione con grandi volumi di dati eterogenei. Implementa l'architettura Medallion (nota anche come architettura multi-hop o Delta) tipica delle moderne data lakehouse, strutturando i dati in layer progressivi (Bronze, Silver e Gold) che garantiscono scalabilità, affidabilità e prestazioni elevate.

Il sistema si distingue per:

- Interfaccia conversazionale basata su AI per interrogazioni in linguaggio naturale
- Pipeline di trasformazione automatizzate tra i layer Bronze, Silver e Gold
- Gestione integrata della qualità e del lineage dei dati
- Scalabilità sia in termini di volume che di varietà dei dati
- Integrazione nativa con Google Cloud Platform (BigQuery, Cloud Storage, Dataplex)

Per maggiori dettagli, il codice sorgente del progetto è disponibile al seguente link GitHub, cliccando sull'icona si viene reindirizzati direttamente al repository.



https://github.com/lukebo01/easy_query

EasyQuery si propone di risolvere diverse problematiche tipiche della gestione dei dati su larga scala:

- Ridurre la complessità nell'accesso ai dati strutturati e non strutturati
- Eliminare la necessità di conoscenza approfondita di SQL per l'interrogazione dei dati
- Automatizzare la trasformazione e la pulizia dei dati
- Garantire la tracciabilità delle trasformazioni (data lineage)
- Ottimizzare le prestazioni per query analitiche complesse
- Scalare orizzontalmente per gestire volumi crescenti di dati

Indice

1	Architettura del Sistema	1
1.1	Architettura Medallion	1
1.1.1	Layer Bronze	1
1.1.2	Layer Silver	2
1.1.3	Layer Gold	2
1.2	Componenti del Sistema	2
1.2.1	Frontend	3
1.2.2	Servizi Core	3
1.2.3	Cloud Functions	4
2	Ingestion e Governance	5
2.1	Processo di Ingestion Automatizzato	5
2.1.1	Caricamento e Trigger	5
2.1.2	Estrazione Arricchita dei Metadati	5
2.2	Data Governance e Discovery con Dataplex	6
2.2.1	Dalla Trasformazione alla Scoperta	6
2.2.2	Scansione Forzata di Dataplex	6
2.2.3	Implicazioni e Latenza del Processo	6
3	Pipeline di Elaborazione	8
3.1	Pipeline Bronze-to-Silver	8
3.1.1	Processo di Trasformazione e per tipo di file	8
3.1.2	Normalizzazione dei dati	10
3.1.3	Scalabilità della pipeline	10
3.2	Pipeline Silver-to-Gold	10
3.2.1	Processo di trasformazione	11
3.2.2	Caratteristiche principali	11
3.2.3	Scalabilità della pipeline	11
4	Funzionalità di Interrogazione Intelligente	12
4.1	Elaborazione del Linguaggio Naturale	12
4.1.1	Processo di Elaborazione	12
4.1.2	Ottimizzazioni NLP	12
4.2	Query Builder	14
4.3	Linee guida per la generazione di SQL	14
5	Scalabilità e Performance	15
5.1	Architettura Medallion per la Scalabilità	15
5.1.1	Scalabilità in Variety (Varietà)	15
5.1.2	Scalabilità in Volume	15
5.2	Ottimizzazioni per Grandi Dataset	16
5.3	Gestione della Scalabilità del Frontend	16
6	Implementazione Tecnica	17
6.1	Stack Tecnologico	17

6.2	Implementazione delle Pipeline	17
6.2.1	Servizio di Orchestrazione	17
6.2.2	Bronze-to-Silver Function	18
6.2.3	Silver-to-Gold Function	19
6.3	Integrazione con Gemini AI	19
6.4	Gestione del Frontend	20
7	Gestione della Scalabilità e Ottimizzazione dei Prompt	22
7.1	Strategie di Batching e Campionamento	22
7.1.1	Selezione dei File Bronze (<code>batchedSuggestBronzeFiles</code>)	22
7.1.2	Discovery di Metadati Silver e Gold (<code>batchedGoldAndSilverDiscovery</code>)	22
7.1.3	Analisi dei Risultati (<code>batchedAnalyzeQueryResults</code>)	23
7.2	Limiti della Strategia e Mitigazioni Future	23
8	Conclusioni	25
8.1	Risultati Raggiunti	25
8.2	Sviluppi Futuri	25
8.3	Considerazioni Finali	25

1 Architettura del Sistema

1.1 Architettura Medallion

EasyQuery implementa l'architettura Medallion, suddividendo i dati in tre layer principali:

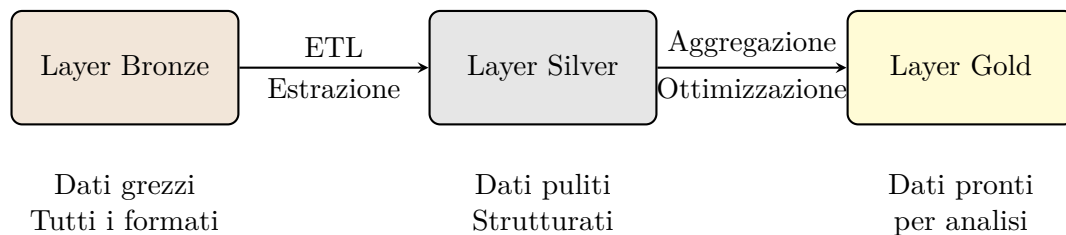


Figura 1.1: Architettura Medallion di EasyQuery

1.1.1 Layer Bronze

Il layer Bronze rappresenta il punto di ingresso di tutti i dati nel sistema. Caratteristiche principali:

- Conserva i dati grezzi nel loro formato originale
- Supporta un'ampia varietà di formati (CSV, JSON, PDF, immagini, ecc.)
- Arricchisce i dati con metadati di ingestion (timestamp, origine, tipo)
- Implementato utilizzando Google Cloud Storage per massima scalabilità
- Tutti i file sono tracciati in una tabella di metadati BigQuery

I metadati vengono registrati nella tabella `metadata_store.bronze_file_metadata`, che include:

- Percorsi GCS dei file originali
- Timestamp di caricamento
- Informazioni sul formato e sulla dimensione
- Stato di elaborazione
- Collegamenti ai dati nel layer Silver

1.1.2 Layer Silver

Il layer Silver contiene dati puliti, trasformati e strutturati:

- Conversione di tutti i dati in formato Parquet per efficienza
- Normalizzazione dei nomi delle colonne (snake_case)
- Inferenza e standardizzazione dei tipi di dati
- Estrazione di testo da documenti PDF e immagini tramite OCR
- Organizzazione in una struttura gerarchica basata su dominio e tipo
- Registrazione dei metadati e del lineage completo

I file Parquet vengono organizzati secondo una logica di dominio e contesto:

```
silver-layer-bucket/files/fin_str/csv/sales/monthly_report.parquet
```

Dove:

- `fin_str`: Dominio finanziario, dati strutturati
- `csv`: Tipo di contenuto originale
- `sales`: Contesto applicativo
- `monthly_report`: Nome del dataset

1.1.3 Layer Gold

Il layer Gold contiene dati ottimizzati per l'analisi:

- Tabelle pre-aggregate pronte per query specifiche
- Ottimizzazione delle strutture dati per performance di interrogazione
- Materializzazione delle viste più utilizzate
- Implementato come tabelle BigQuery
- Metadati completi su origine e trasformazioni applicate

1.2 Componenti del Sistema

EasyQuery è composto da diverse componenti integrate:

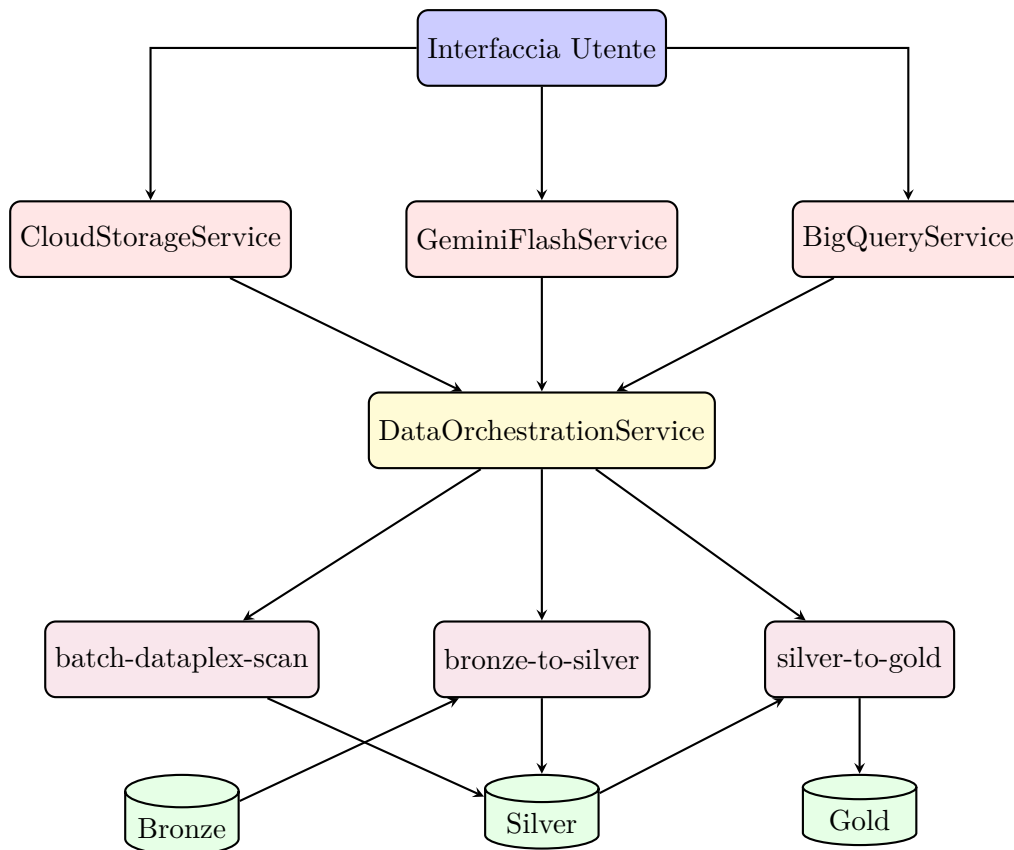


Figura 1.2: Architettura delle componenti di EasyQuery

1.2.1 Frontend

Implementato in Flutter, offre:

- Interfaccia utente responsive e multi-piattaforma
- Box di ricerca per query in linguaggio naturale
- Funzionalità di caricamento intelligente dei file
- Visualizzazione avanzata dei risultati con grafici interattivi
- Modalità di esportazione e condivisione dei risultati

1.2.2 Servizi Core

- **GeminiFlashService:** Integrazione con Google Gemini per elaborazione del linguaggio naturale e generazione di SQL
- **BigQueryService:** Gestione delle connessioni e delle query verso BigQuery
- **CloudStorageService:** Interazione con Google Cloud Storage per la gestione dei file
- **DataOrchestrationService:** Orchestrazione delle pipeline di elaborazione dati

1.2.3 Cloud Functions

- **ingestion:** Attivata da un upload nel layer Bronze, estrae e arricchisce i metadati analizzando il contenuto del file (es. testo da PDF, EXIF da immagini) e lo registra nel catalogo BigQuery per la successiva elaborazione.
- **bronze-to-silver:** Trasforma i dati dal formato grezzo a Parquet strutturato.
- **silver-to-gold:** Materializza tabelle ottimizzate per analisi.
- **batch-dataplex-scan:** Aggiorna i metadati e mantiene aggiornata la Discovery di Dataplex.

2 Ingestion e Governance

Un aspetto fondamentale di EasyQuery è la sua capacità di gestire l'intero ciclo di vita del dato, dall'ingestion iniziale fino alla sua governance e disponibilità per l'analisi. Questo processo è orchestrato tramite un'architettura event-driven basata su Cloud Functions e servizi di governance come Dataplex.

2.1 Processo di Ingestion Automatizzato

L'ingestion dei dati è il primo passo per rendere le informazioni disponibili all'interno del sistema. Il processo è progettato per essere il più automatico e intelligente possibile.

2.1.1 Caricamento e Trigger

Il processo inizia quando un utente carica un file tramite l'interfaccia Flutter. Il file non viene elaborato direttamente, ma viene inviato al bucket di Google Cloud Storage (GCS) designato come **layer Bronze**. La creazione di un nuovo oggetto (file) in questo bucket funge da evento scatenante (trigger) che invoca automaticamente una Cloud Function specifica: `gcs_metadata_extractor`.

2.1.2 Estrazione Arricchita dei Metadati

La funzione `gcs_metadata_extractor` è il cuore dell'ingestion intelligente. Il suo scopo non è solo registrare l'esistenza di un file, ma arricchirlo con metadati contestuali per facilitarne la successiva scoperta e trasformazione. Il suo funzionamento si articola in più fasi:

1. **Identificazione di Base:** La funzione estrae i metadati standard forniti da GCS, come il nome del file, il percorso, la dimensione, l'URI e il tipo di contenuto (MIME type).
2. **Analisi del Contenuto:** A differenza di un semplice catalogo, la funzione ispeziona il contenuto del file per estrarre informazioni più profonde, adattando il suo comportamento al tipo di file:
 - **PDF:** Utilizza la libreria `PyPDF2` per estrarre il testo completo e contare il numero di pagine.
 - **Immagini (JPG, PNG, etc.):** Utilizza la libreria `Pillow` per identificare metadati specifici come formato, dimensioni in pixel e dati EXIF, se presenti.
 - **File Testuali (TXT, CSV, JSON):** Legge una porzione iniziale del contenuto per l'analisi, evitando di caricare in memoria file di grandi dimensioni.
3. **Inferenza Semantica:** Sfruttando il testo estratto e il percorso del file, la funzione tenta di comprendere il significato dei dati:

- **Tagging Automatico:** Analizza il testo alla ricerca di parole chiave predefinite per assegnare tag di categoria (es. `finance`, `human_resources`, `legal`). Questo aiuta a classificare i dati in modo trasversale.
 - **Dominio dei Dati:** Cerca di inferire un dominio di appartenenza (es. `financial`, `marketing`, `operations`) basandosi su parole chiave presenti nel percorso del file o nel suo contenuto.
4. **Persistenza dei Metadati:** Tutte le informazioni raccolte (base, estratte e inferite) vengono consolidate in un record JSON e inserite in una riga della tabella BigQuery `metadata_store.bronze_file_metadata`. Questa tabella agisce come un catalogo centrale e arricchito di tutti i dati grezzi presenti nel sistema, contrassegnando il file come `AVAILABLE_IN_BRONZE`.

2.2 Data Governance e Discovery con Dataplex

Dopo che la pipeline `bronze-to-silver` ha trasformato i file grezzi in file Parquet strutturati, questi devono essere resi visibili e interrogabili come tabelle in BigQuery. Questo processo è gestito tramite Google Cloud Dataplex, il servizio di data fabric intelligente di Google Cloud.

2.2.1 Dalla Trasformazione alla Scoperta

I file Parquet generati nel layer Silver sono, inizialmente, solo file all'interno di un bucket GCS. Affinché BigQuery possa interrogarli, è necessario creare delle "tabelle esterne" che puntino a questi file. La creazione manuale di queste tabelle sarebbe inefficiente e soggetta a errori.

2.2.2 Scansione Forzata di Dataplex

Per automatizzare questo processo, EasyQuery invoca la Cloud Function `batch-dataplex-scan`. Questa funzione ha un unico scopo: avviare programmaticamente un job di scansione di Dataplex sull'asset GCS che corrisponde al layer Silver. Dataplex esegue le seguenti operazioni:

- **Scansiona** i percorsi GCS specificati.
- **Rileva** automaticamente i nuovi file Parquet.
- **Inferisce** lo schema dei dati (colonne e tipi di dato) leggendo i metadati interni dei file Parquet.
- **Crea o aggiorna** le tabelle esterne corrispondenti nel dataset BigQuery associato, rendendole disponibili per le query SQL.

2.2.3 Implicazioni e Latenza del Processo

Questo approccio garantisce una governance centralizzata e automatizzata, ma introduce una latenza da considerare. Il processo di scansione di Dataplex è asincrono e non istantaneo. A seconda del carico e del numero di file, può richiedere tra i **6 e i 10 minuti** per essere completato.

Questa latenza ha un'implicazione diretta sull'esperienza utente: dopo la trasformazione di un file in formato Silver, il sistema (e quindi l'utente) deve attendere il completamento della scansione

di Dataplex prima che i nuovi dati siano visibili e possano essere inclusi nelle interrogazioni in linguaggio naturale. Si tratta di un compromesso architetturale consapevole, che privilegia l'automazione e la robustezza della governance rispetto alla disponibilità immediata dei dati.

3 Pipeline di Elaborazione

3.1 Pipeline Bronze-to-Silver

La pipeline Bronze-to-Silver rappresenta uno dei processi core di EasyQuery, responsabile della trasformazione dei dati grezzi in formati strutturati e ottimizzati.

3.1.1 Processo di Trasformazione e per tipo di file

La pipeline implementa strategie specifiche per ogni tipo di file:

File testuali (CSV, JSON, TXT)

- Rilevamento automatico dei delimitatori per CSV
- Parsing intelligente del formato (tabellare vs dizionario) per TXT
- Gestione di JSON annidati e JSONL
- Validazione e pulizia dei dati
- Inferenza automatica dei tipi di colonna

Documenti PDF

- Rendering ad alta risoluzione delle pagine con PyMuPDF
- OCR avanzato tramite Cloud Vision API
- Estrazione della struttura del documento
- Conversione del testo estratto in formato tabulare

Immagini

- OCR con Vision API per estrarre il testo
- Estrazione di metadati EXIF
- Rilevamento di informazioni contestuali
- Conversione in struttura tabulare

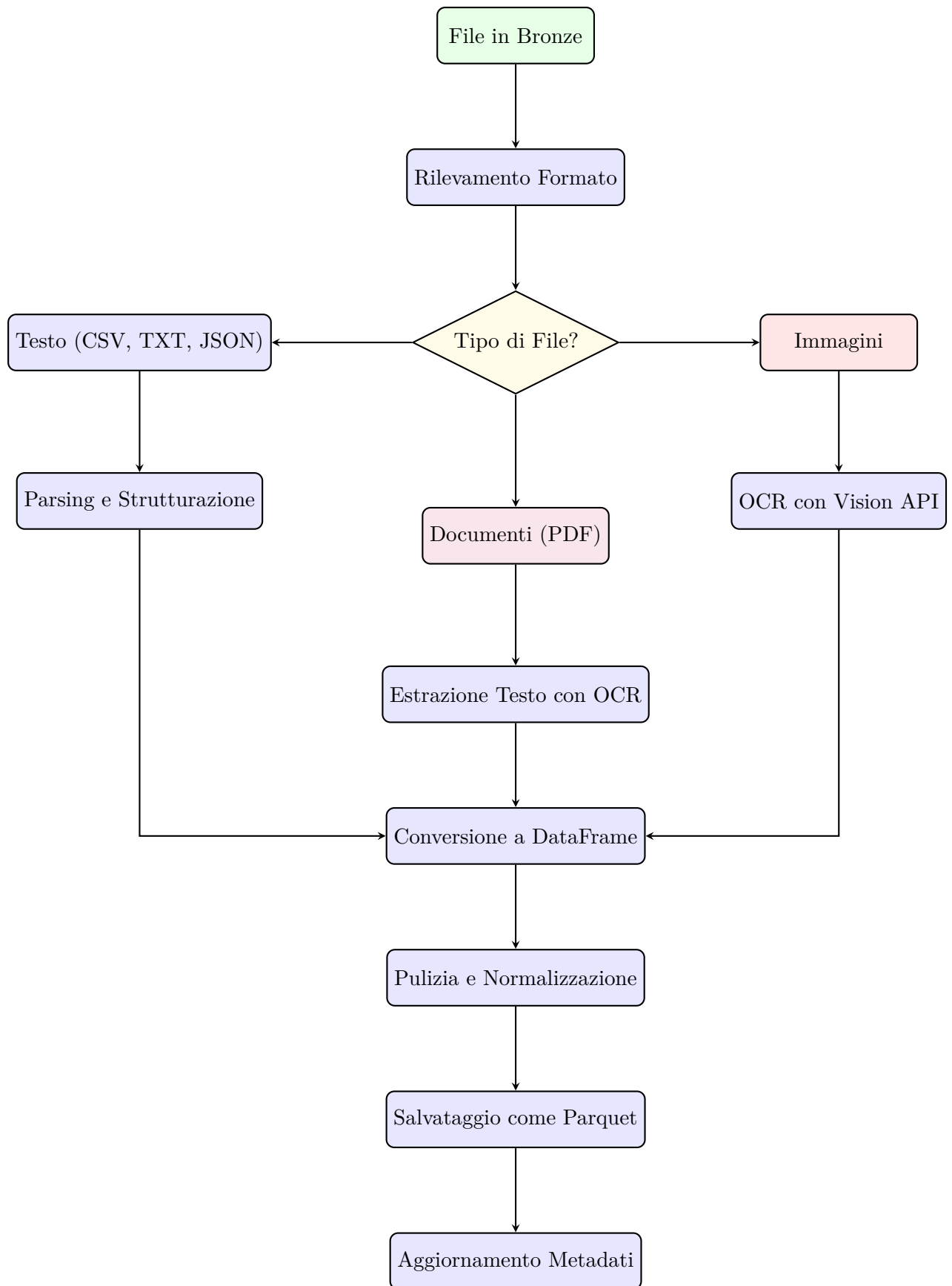


Figura 3.1: Processo di trasformazione Bronze-to-Silver (rivisto)

3.1.2 Normalizzazione dei dati

Per tutti i tipi di file, vengono applicati processi di normalizzazione:

- Standardizzazione dei nomi di colonna in snake_case
- Rimozione di caratteri speciali e spazi
- Inferenza e coercizione dei tipi di dati
- Gestione di valori nulli e mancanti
- Generazione di colonne aggiuntive di metadati

3.1.3 Scalabilità della pipeline

La pipeline Bronze-to-Silver è progettata per scalare su diversi assi:

Scalabilità in volume

- Elaborazione in stream con buffer temporanei
- Evita caricamento completo di file di grandi dimensioni
- Utilizzo di Pandas con allocazione dinamica della memoria
- Conversione efficiente in formato Parquet con compressione

Scalabilità in varietà

- Supporto per più di 20 formati di file diversi
- Framework estensibile per aggiungere nuovi parser
- Rilevamento automatico del formato
- Strategie di fallback per formati sconosciuti

Implementazione serverless

- Esecuzione come Cloud Function per scalabilità orizzontale automatica
- Recupero automatico da errori temporanei
- Timeout configurabili per file di grandi dimensioni
- Logging dettagliato per debugging e monitoraggio

3.2 Pipeline Silver-to-Gold

La pipeline Silver-to-Gold rappresenta il processo di trasformazione dei dati strutturati in formato Silver in tabelle ottimizzate per l'analisi nel layer Gold.

3.2.1 Processo di trasformazione

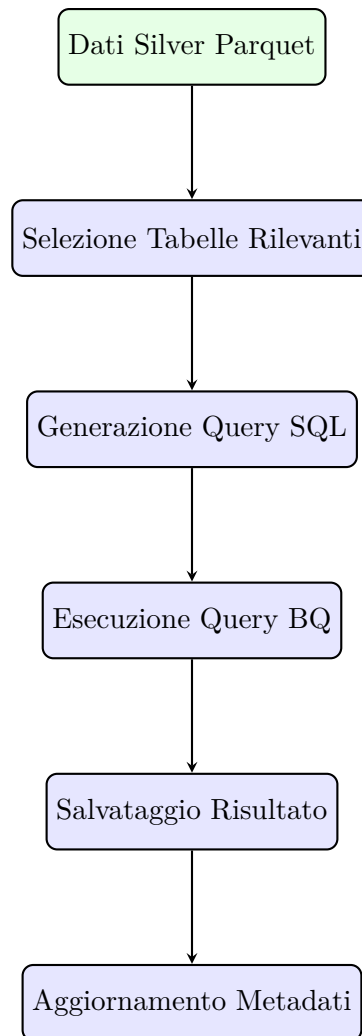


Figura 3.2: Processo di trasformazione Silver-to-Gold

3.2.2 Caratteristiche principali

- Esecuzione di query SQL direttamente su BigQuery per performance
- Possibilità di definire viste materializzate
- Controlli di sicurezza per evitare query dannose
- Registro completo delle trasformazioni applicate

3.2.3 Scalabilità della pipeline

- Utilizzo della potenza di elaborazione di BigQuery
- Partizionamento automatico delle tabelle
- Clustering basato sulle colonne più frequentemente interrogate
- Ottimizzazione delle query SQL generate

4 Funzionalità di Interrogazione Intelligente

4.1 Elaborazione del Linguaggio Naturale

EasyQuery utilizza il modello Google Gemini per trasformare domande in linguaggio naturale in query SQL eseguibili:

4.1.1 Processo di Elaborazione

1. **Normalizzazione e traduzione:** La domanda viene tradotta in inglese per massimizzare le performance dell'LLM
2. **Estrazione dell'intento:** Viene identificato lo scopo principale della query
3. **Recupero dei metadati:** Il sistema recupera schemi, campioni e statistiche dei dati disponibili
4. **Selezione delle tabelle:** Vengono identificate le tabelle più rilevanti per la query
5. **Generazione SQL:** Il modello genera la query SQL ottimizzata
6. **Esecuzione e analisi:** La query viene eseguita e i risultati analizzati
7. **Presentazione:** I risultati vengono formattati e presentati all'utente

4.1.2 Ottimizzazioni NLP

- **Preprocessing dei prompt:** Strutturazione dei prompt per massimizzare l'efficacia del modello
- **Contestualizzazione:** Inclusione di esempi di tabelle e schema nel prompt
- **Prompt engineering:** Tecniche avanzate per guidare il modello
- **Raffinamento iterativo:** Le query sono testate e raffinate automaticamente
- **Fallback strategies:** Meccanismi per gestire ambiguità e richieste poco chiare

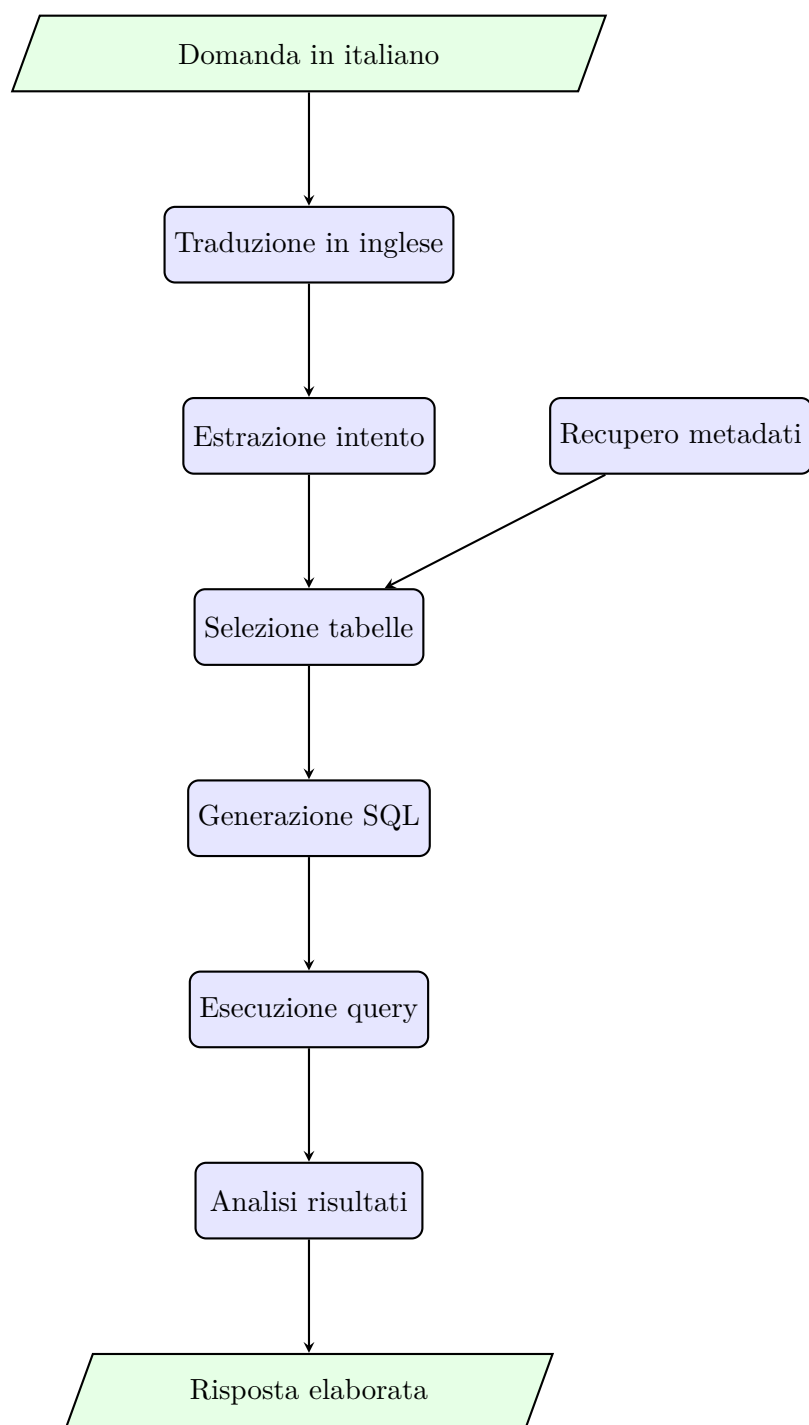


Figura 4.1: Pipeline di elaborazione delle domande in linguaggio naturale

4.2 Query Builder

Il componente Query Builder è responsabile della generazione di SQL ottimizzato:

- Selezione intelligente delle tabelle basata sulla rilevanza per l'intento dell'utente
- Prioritizzazione delle tabelle recentemente create nel processo Bronze-to-Silver
- Inclusione automatica di join necessari tra tabelle correlate
- Aggiunta di clausole WHERE appropriate basate sull'intento
- Ottimizzazione delle performance con limiti appropriati e suggerimenti per l'ottimizzazione
- Sanitizzazione per prevenire SQL injection e altri problemi di sicurezza

4.3 Linee guida per la generazione di SQL

Il sistema applica diverse strategie per garantire la qualità del SQL generato:

- Preferenza per aggregazioni sul lato server per ridurre il trasferimento dati
- Uso appropriato di funzioni analitiche per calcoli complessi
- Strategie di ottimizzazione basate sulle statistiche delle tabelle
- Limiti di sicurezza per il numero di record restituiti
- Gestione degli errori di sintassi e semantici
- Logging dettagliato delle query generate per debugging

5 Scalabilità e Performance

5.1 Architettura Medallion per la Scalabilità

L'adozione dell'architettura Medallion fornisce diversi vantaggi in termini di scalabilità:

5.1.1 Scalabilità in Variety (Varietà)

- **Bronze Layer:** Accetta qualsiasi formato di dati senza restrizioni
- **Silver Layer:** Standardizza i dati in un formato comune (Parquet)
- **Gold Layer:** Ottimizza i dati per specifici pattern di interrogazione

La separazione in layer permette di:

- Aggiungere nuovi tipi di sorgenti dati senza modificare l'intero sistema
- Mantenere la coerenza dello schema nei layer superiori indipendentemente dai formati di input
- Applicare trasformazioni specifiche per tipo di dato al livello appropriato

5.1.2 Scalabilità in Volume

- **Bronze Layer:** Utilizza Google Cloud Storage per scalare a petabyte di dati
- **Silver Layer:**
 - Organizzazione gerarchica per domain/context
 - Compressione efficiente con Parquet
 - Processamento distribuito con Cloud Functions
- **Gold Layer:** Sfrutta BigQuery per:
 - Query distribuite su migliaia di nodi
 - Partizionamento e clustering automatico
 - Separazione storage/compute
 - Auto-scaling delle risorse

5.2 Ottimizzazioni per Grandi Dataset

- **Chunking:** Suddivisione di operazioni pesanti in batch più piccoli
- **Sampling intelligente:** Analisi su campioni rappresentativi per operazioni di discovery
- **Lazy loading:** Caricamento dei dati solo quando necessario
- **Query planning:** Ottimizzazione automatica delle query basata sulla struttura dei dati

5.3 Gestione della Scalabilità del Frontend

La gestione della scalabilità a livello di frontend presenta sfide uniche, in quanto l'applicazione client (Flutter) opera con risorse di memoria e calcolo limitate, specialmente su dispositivi mobili o browser web. Il caricamento di un set di risultati potenzialmente molto esteso, proveniente da una query su BigQuery, potrebbe facilmente portare a un consumo eccessivo di memoria, causando blocchi dell'interfaccia utente o crash dell'applicazione (errore OutOfMemory). Per garantire un'esperienza utente fluida e reattiva, è stata adottata una strategia pragmatica di limitazione dei dati visualizzati. Invece di implementare meccanismi complessi di paginazione o rendering virtualizzato, che non risolverebbero il problema del caricamento iniziale di un grande volume di dati in memoria, si è scelto di agire direttamente sulla fonte dei dati. Nello specifico, a ogni query SQL generata dal sistema e destinata a BigQuery viene automaticamente aggiunta una clausola LIMIT 100. Questa scelta architetturale comporta i seguenti vantaggi e implicazioni:

- **Responsività e Stabilità Garantite:** Previene il sovraccarico della memoria del client, assicurando che l'applicazione rimanga sempre stabile e reattiva, indipendentemente dalla dimensione totale del set di risultati della query.
- **Visualizzazione come Anteprima:** All'utente viene presentata un'anteprima significativa e immediata dei primi 100 record del risultato. Questo è sufficiente nella maggior parte dei casi per validare la correttezza della query e comprendere la struttura dei dati restituiti.
- **Separazione tra Analisi e Dati Grezzi:** È fondamentale sottolineare che questa limitazione si applica esclusivamente alla visualizzazione dei dati tabellari nel frontend. L'analisi testuale generata da Gemini, al contrario, viene eseguita su un campione più ampio e rappresentativo del dataset (come descritto nel capitolo sulla gestione della scalabilità dei prompt), garantendo così che l'insight qualitativo non sia compromesso dalla limitazione imposta all'interfaccia.

6 Implementazione Tecnica

6.1 Stack Tecnologico

Componente	Tecnologia	Scopo
Frontend	Flutter	UI multi-piattaforma
Backend	Google Cloud Functions	Servizi serverless
AI	Google Gemini	NLP e generazione SQL
Storage primario	Google Cloud Storage	Storage layer Bronze
Database analitico	BigQuery	Layer Silver e Gold
Metadati	Dataplex	Catalogo e governance
Elaborazione dati	Pandas, PyArrow	Trasformazione dati
OCR	Cloud Vision API	Estrazione testo da immagini e PDF

Tabella 6.1: Stack tecnologico di EasyQuery

6.2 Implementazione delle Pipeline

6.2.1 Servizio di Orchestrazione

La classe `DataOrchestrationService` è il componente centrale che coordina l'intero processo:

```
1 class DataOrchestrationService {
2   final GeminiFlashService _geminiService;
3   final BigQueryService _bigQueryService;
4   final CloudStorageService _cloudStorageService;
5
6   // URL delle Cloud Functions
7   final String _bronzeToSilverUrl;
8   final String _silverToGoldUrl;
9   final String _batchDataplexScanUrl;
10
11  // ... costruttore e metodi di inizializzazione ...
12
13  Future<Map<String, dynamic>> dataTransformationPipeline(
14    String userIntent,
15    List<Map<String, dynamic>> schemas,
16    Map<String, List<Map<String, dynamic>>> sampleData,
17    List<Map<String, dynamic>> bronzeFilesMetadata
18  ) async {
19    // Pipeline completa di trasformazione dati
20    // ...
21  }
22
23  Future<String> queryBuildingPipeline(
24    String userIntent,
25    Map<String, dynamic> selectedSchemas
```

```

26 ) async {
27     // Pipeline di generazione SQL
28     // ...
29 }
30 }

```

Listing 6.1: Implementazione DataOrchestrationService

6.2.2 Bronze-to-Silver Function

La Cloud Function `bronze-to-silver` implementa la logica di trasformazione:

```

1 @functions_framework.http
2 def bronze_to_silver(request: Request):
3     # Validazione input
4     # ...
5
6     # Estrazione percorso file bronze
7     bronze_bucket_name = parts[0]
8     blob_name_in_gcs = '/' + parts[1]
9
10    # Download del file in un percorso temporaneo
11    with tempfile.NamedTemporaryFile(delete=False) as tmp_in:
12        tmp_in_path = tmp_in.name
13
14    bronze_blob.download_to_filename(tmp_in_path)
15
16    # Determina i componenti del percorso silver
17    path_suffix_components = determine_silver_path_components(
18        blob_name_in_gcs,
19        file_extension_original,
20        original_blob_content_type
21    )
22
23    # Elabora il file in base al tipo
24    df_processed = process_file_by_type(
25        tmp_in_path,
26        file_extension_original,
27        original_blob_content_type,
28        original_blob_size,
29        file_name_original_ext
30    )
31
32    # Prepara DataFrame e salva come Parquet
33    df_data_only, metadata_values = prepare_dataframe_for_parquet(df_processed,
34        metadata_columns)
35    save_dataframe_to_parquet(df_data_only, tmp_out_path)
36
37    # Carica su GCS e aggiorna manifest
38    uri_parquet_in_silver = upload_to_gcs_and_update_manifest(...)
39
40    # Aggiorna metadati
41    update_bronze_metadata_in_bigquery(...)
42
43    return response_data

```

Listing 6.2: Implementazione bronze-to-silver

6.2.3 Silver-to-Gold Function

La Cloud Function `silver-to-gold` implementa la materializzazione dei dati analitici:

```

1 @functions_framework.http
2 def silver_to_gold(request: Request):
3     # Validazione input
4     # ...
5
6     # Estrai parametri richiesti
7     query = data_payload.get("query")
8     output_name = data_payload.get("output_name")
9
10    # Esegui la query e salva i risultati
11    result = execute_query_and_save_as_table(
12        query,
13        output_name,
14        current_project_id,
15        GOLD_BQ_DATASET_CFG
16    )
17
18    # Aggiungi metadati alla risposta
19    result["message"] = f"Query results saved successfully to table {result['gold_bigquery_table']}"
20    result["optimization_type"] = "query_result"
21    result["description"] = description
22    result["timestamp"] = datetime.datetime.utcnow().isoformat()
23
24    return result

```

Listing 6.3: Implementazione silver-to-gold

6.3 Integrazione con Gemini AI

Il servizio `GeminiFlashService` gestisce l'integrazione con l'AI:

```

1 class GeminiFlashService {
2     final RestService _restService;
3     final String _apiKey;
4     final String _baseUrl = "https://generativelanguage.googleapis.com/v1/models"
5     ;
6     final String _model = "gemini-flash-2.0";
7
8     // ... costruttore e metodi di utilità ...
9
10    Future<String> translateToEnglish(String text) async {
11        // Traduzione del testo in inglese
12        // ...
13    }
14
15    Future<String> getUserIntent(String question) async {
16        // Estrazione dell'intento dalla domanda
17        // ...
18    }
19
20    Future<Map<String, List<String>>> suggestRelevantTables(
21        String intent,
22        List<String> tableNames,
23        List<Map<String, dynamic>> schemas,
24        Map<String, List<Map<String, dynamic>>> sampleData,

```

```

24     List<String> recentlyCreatedTables
25   ) async {
26     // Suggerimento tabelle rilevanti per l'intento
27     // ...
28   }
29
30   Future<String> generateSqlQuery(
31     String intent,
32     Map<String, dynamic> tableSchemas,
33     Map<String, List<Map<String, dynamic>>> sampleData
34   ) async {
35     // Generazione della query SQL
36     // ...
37   }
38
39   Future<String> analyzeQueryResults(
40     String query,
41     List<Map<String, dynamic>> results
42   ) async {
43     // Analisi dei risultati della query
44     // ...
45   }
46 }

```

Listing 6.4: Implementazione GeminiFlashService

6.4 Gestione del Frontend

L'interfaccia utente è implementata in Flutter con un approccio orientato allo stato:

```

1  class _SearchPageState extends State<SearchPage> {
2    // ... stato e controller ...
3
4    Future<void> _processQuestion(String question) async {
5      // Reset esplicito delle liste per ogni nuova query
6      List<Map<String, dynamic>> cloudFilesMetadata = [];
7      List<Map<String, dynamic>> schemas = [];
8      List<String> tableNames = [];
9      Map<String, List<Map<String, dynamic>>> sampleData = {};
10
11      // Aggiorna lo stato per indicare che l'elaborazione è iniziata
12      setState(() {
13        _isLoading = true;
14        _errorMessage = '';
15        _currentExecutingQuery = 'Starting analysis...';
16      });
17
18      try {
19        // Traduci la domanda in inglese
20        question = await widget.geminiService.translateToEnglish(question);
21
22        // Recupera i metadati bronze
23        cloudFilesMetadata = await widget.bigQueryService.getBronzeMetadata();
24
25        // Recupera le tabelle Silver e Gold
26        Map<String, List<String>> datasetTablesMap =
27          await dataOrchestrationService.getSilverAndGoldTables();
28
29        // Ottieni nomi delle tabelle, schemi e i dati di esempio
30        final schemaData = await dataOrchestrationService.initializeTableMaps(
31          datasetTablesMap,

```

```

32     );
33
34     // Ottieni l'intento dell'utente dalla domanda
35     String userIntent = await dataOrchestrationService.getUserIntent(question
36     );
37
38     // Dai il via al processo di data transformation
39     Map<String, dynamic> selectedAndUpdatedSchemasMap =
40         await dataOrchestrationService.dataTransformationPipeline(...);
41
42     // Dai il via al processo di creazione della query
43     String sqlQuery = await dataOrchestrationService.queryBuildingPipeline
44         (...);
45
46     // Esegui la query pulita su BQ
47     final results = await widget.bigQueryService.executeQuery(cleanedSqlQuery
48     );
49
50     // Effettua l'analisi dei risultati ottenuti
51     final analysis = await widget.geminiService.batchedAnalyzeQueryResults
52         (...);
53
54     // Vai alla pagina dei risultati
55     Navigator.push(context, MaterialPageRoute(...));
56 } catch (e, stackTrace) {
57     // Gestione errori
58 } finally {
59     // Ripulisci lo stato
60 }
61
62 // ... altri metodi UI ...
63 }

```

Listing 6.5: Implementazione SearchPage

7 Gestione della Scalabilità e Ottimizzazione dei Prompt

L'architettura di EasyQuery è progettata per gestire grandi volumi di dati, ma la sua componente di intelligenza artificiale, basata su Google Gemini, introduce una sfida specifica: il limite della finestra di contesto. I modelli linguistici di grandi dimensioni (LLM) possono processare solo una quantità finita di informazioni in una singola richiesta, nota come "limite di token". Per il modello `gemini-flash-2.0`, questo limite è di circa un milione di token.

In uno scenario con migliaia di file e tabelle, inviare tutti i metadati (schemi, nomi di file, dati di esempio) in un'unica richiesta a Gemini è impraticabile e porterebbe a un errore di superamento del limite. Per risolvere questo problema, EasyQuery implementa strategie avanzate di **batching** (suddivisione in lotti) e **campionamento**, che garantiscono la scalabilità del sistema mantenendo l'interazione con l'AI efficiente e funzionale.

7.1 Strategie di Batching e Campionamento

Sono state sviluppate tre diverse strategie, ognuna ottimizzata per una fase specifica del processo di interrogazione, per gestire la complessità e la dimensione dei dati inviati a Gemini.

7.1.1 Selezione dei File Bronze (`batchedSuggestBronzeFiles`)

Durante la fase di trasformazione dati, è necessario identificare quali file nel layer Bronze sono rilevanti per l'intento dell'utente e necessitano di essere processati nel layer Silver.

- **Creazione dei lotti:** Il sistema prima stima se il prompt completo con tutti i metadati dei file Bronze supererebbe il limite di token. Se così fosse, i metadati vengono suddivisi in lotti di 50 elementi.
- **Suddivisione ricorsiva:** Se anche un lotto di 50 elementi risulta troppo grande, viene suddiviso ricorsivamente a metà fino a quando la sua dimensione non rientra nei limiti.
- **Aggregazione dei risultati:** Ogni lotto viene inviato a Gemini in una richiesta separata. I nomi dei file suggeriti da ogni chiamata vengono raccolti in un **Set**, una struttura dati che garantisce l'eliminazione automatica di eventuali duplicati. Il risultato finale è una lista unica e consolidata di file da processare.

7.1.2 Discovery di Metadati Silver e Gold (`batchedGoldAndSilverDiscovery`)

Una volta che i dati sono disponibili nei layer Silver e Gold, il sistema deve selezionare le tabelle più pertinenti per rispondere alla domanda dell'utente. Questa fase è più complessa, poiché coinvolge schemi di tabelle e dati di esempio.

- **Batching intelligente:** Gli schemi delle tabelle vengono suddivisi in lotti di 10. Per ogni lotto di schemi, il sistema estrae in modo mirato solo i dati di esempio corrispondenti a quelle tabelle, limitando ogni campione a un massimo di 5 righe per contenere la dimensione del prompt.
- **Gestione ricorsiva:** Similmente alla strategia precedente, se un lotto combinato di schemi e campioni supera il limite, viene suddiviso ricorsivamente.
- **Aggregazione separata:** I risultati vengono aggregati in due Set distinti, uno per le tabelle Silver suggerite e uno per quelle Gold. Questo permette di mantenere la distinzione tra i due layer e di utilizzare in modo prioritario le tabelle Gold, già ottimizzate per l'analisi.

7.1.3 Analisi dei Risultati (`batchedAnalyzeQueryResults`)

Dopo l'esecuzione di una query SQL, i risultati possono essere molto voluminosi (migliaia o milioni di righe). Inviare l'intero dataset a Gemini per un'analisi testuale è infattibile. L'approccio qui è diverso e si basa sul campionamento.

- **Campionamento distribuito:** Invece di creare lotti sequenziali, il sistema adotta una strategia di campionamento rappresentativo. Se il set di risultati è troppo grande, vengono estratte 100 righe distribuite in modo strategico:
 - Un terzo delle righe dall'inizio del dataset.
 - Un terzo dalla parte centrale.
 - Un terzo dalla fine del dataset.
- **Analisi su campione:** Questo campione, pur essendo di dimensioni ridotte, offre una visione rappresentativa dell'intero set di dati. Viene inviato a Gemini in una singola chiamata per generare un'analisi sintetica.
- **Trasparenza per l'utente:** All'utente viene comunicato esplicitamente che l'analisi testuale è basata su un campione dei risultati, garantendo la correttezza dell'interpretazione.

7.2 Limiti della Strategia e Mitigazioni Future

Le strategie di batching implementate presentano una vulnerabilità teorica: la possibilità che un **singolo elemento indivisibile** (ad esempio, lo schema di una tabella estremamente complessa o i metadati di un singolo file molto articolato) possa da solo superare il limite di token. Nei metodi attuali, la suddivisione ricorsiva si ferma a un numero minimo di elementi (es. 2 o 10), ma non gestisce il caso in cui un singolo elemento sia il problema.

Questo rappresenta un caso limite ma possibile in scenari con dati particolarmente annidati o eterogenei. Per affrontare questa vulnerabilità, gli sviluppi futuri potrebbero includere:

- **Semplificazione selettiva:** Algoritmi per ridurre la complessità di un singolo schema troppo grande, ad esempio rimuovendo dettagli non essenziali o troncando descrizioni lunghe.
- **Riassunto dei metadati:** Utilizzo di un modello AI per generare un riassunto conciso di un singolo elemento troppo verboso prima di includerlo nel prompt principale.

- **Tecniche di embedding:** Conversione di schemi e metadati in rappresentazioni vettoriali (embedding) per eseguire una ricerca di similarità più efficiente e meno dispendiosa in termini di token.

Queste ottimizzazioni renderebbero il sistema ancora più robusto e capace di gestire anche i casi di dati più estremi.

8 Conclusioni

8.1 Risultati Raggiunti

EasyQuery rappresenta un significativo avanzamento nell'ambito degli strumenti di data analytics, offrendo:

- Interfaccia conversazionale avanzata per query complesse.
- Pipeline automatizzate per la trasformazione dei dati, a partire da un'ingestion intelligente e governata.
- Architettura scalabile che supporta volumi di dati crescenti, gestendo le limitazioni delle API esterne tramite strategie di batching e campionamento.
- Gestione unificata di dati strutturati e non strutturati.
- Visualizzazioni intelligenti dei risultati di analisi.

8.2 Sviluppi Futuri

Le direzioni di evoluzione future includeranno:

- Supporto per data mining avanzato e suggerimenti proattivi.
- Integrazione con sistemi di streaming per dati in tempo reale.
- Funzionalità collaborative per team di analisti.
- Implementazione di strategie di mitigazione per la gestione di singoli elementi di metadati di grandi dimensioni.
- Ottimizzazione dei tempi di latenza del processo di discovery, possibilmente esplorando alternative alla scansione batch di Dataplex.

8.3 Considerazioni Finali

EasyQuery dimostra come l'integrazione di tecnologie AI avanzate con architetture data lakehouse moderne possa democratizzare l'accesso ai dati, riducendo significativamente la barriera tecnica per l'analisi dei dati su larga scala. L'adozione dell'architettura Medallion, abbinata a un solido processo di ingestion e governance automatizzata, ha fornito un framework robusto per gestire la complessità e la scalabilità, mentre le strategie di ottimizzazione dei prompt hanno permesso di superare i limiti intrinseci dei modelli linguistici, garantendo che il sistema possa evolvere mantenendo al contempo robustezza e prestazioni.